

Smart Water Monitoring System

An End-to-End IoT Solution Powered by ESP32, Water-Level Sensors & Cloud MQTT Broker

Prepared By: Deepak K

Domain: Embedded & IoT

Submission Date: 25/09/2026

1. Preface

This report outlines the comprehensive project documentation for the Smart Water Monitoring System. It applies embedded sensing, Wi-Fi connectivity, MQTT communication, and real-time telemetry to monitor water level, flow, tank status, and abnormal consumption.

The project connects core embedded design concepts with practical sensor integration, firmware development, cloud communication, and system validation.

2. Introduction

2.1 Project Background

Modern water management requires efficient digital solutions focused on sustainability, operational efficiency, and automated monitoring. This system leverages IoT products, remote monitoring, and scalable telemetry platforms.

2.2 Objective of this Project

- Gain practical experience with embedded hardware and IoT protocols.
- Design a low-cost water monitoring and telemetry system.
- Develop sensor interfacing, firmware, debugging, and data communication skills.
- Create technical documentation and a structured project repository.

2.3 References

- ESP32 Technical Reference Manual, Espressif Systems.
- MQTT Version 3.1.1 OASIS Standard Specification for IoT Messaging.
- Water-level and flow-sensor interfacing application notes.

2.4 Glossary

IoT: Internet of Things

ESP32: 32-bit Wi-Fi/Bluetooth microcontroller

MQTT: Lightweight publish/subscribe messaging protocol

GPIO: General Purpose Input/Output

ADC: Analog-to-Digital Converter

Water Level: Measured tank fill condition

Flow Rate: Volume of water passing through a pipe per unit time.

3. Problem Statement

Water tanks in homes, institutions, and small facilities are often monitored manually. This can lead to overflow, unnoticed low-water conditions, unnecessary pump operation, and difficulty identifying abnormal water usage. The assigned problem is to design a low-cost, secure, responsive Smart Water Monitoring System that measures tank level and water flow, transmits telemetry over Wi-Fi, and provides alerts or automation based on configurable thresholds.

4. Existing and Proposed Solution

Existing Solutions & Limitations

- Manual tank checking requires regular human intervention.
- Basic float switches generally provide only simple ON/OFF control.
- Standalone indicators do not provide historical or remote telemetry.
- Proprietary systems can be costly and difficult to customize.

Proposed Solution

The proposed Smart Water Monitoring System uses an ESP32 as an edge controller connected to a non-contact/ultrasonic water-level sensor, a water-flow sensor, and optional pump-control relay. Sensor values are processed locally and published to an MQTT broker for dashboard monitoring. Threshold logic can identify overflow risk, low-water conditions, and abnormal flow.

Code & Project Submission Links: Project repository can contain the firmware and documentation files.

5. Proposed Design / Model

5.1 High Level Architecture (HLD)

SENSING LAYER → Water Level Sensor + Flow Sensor → PROCESSING LAYER → ESP32 + Threshold Logic →
CLOUD INTERFACE → Wi-Fi + MQTT Broker → PRESENTATION LAYER → Web/Mobile Dashboard

Figure 5.1: High-Level End-to-End Smart Water Monitoring Architecture

5.2 Low Level Diagram (LLD) & Circuit Interfacing

Hardware Component	ESP32 Pin	Interface/Type	Purpose
Ultrasonic Water-Level Sensor	GPIO 5 / 18	Digital Trigger/Echo	Tank level measurement
Water Flow Sensor	GPIO 27	Digital pulse input	Flow-rate & usage monitoring
Pump Relay Module	GPIO 26	Digital output	Pump control
Status LED	GPIO 2	Digital output	System/status indication

The exact GPIO mapping can be adjusted to the selected sensor modules and prototype wiring. For ultrasonic sensing, the controller calculates tank level from the measured distance; for the flow sensor, pulse frequency is converted into an estimated flow rate.

5.3 System Execution Flowchart

[System Power ON] → [Initialize GPIOs & Sensors] → [Connect to Wi-Fi] → [Connect to MQTT Broker] → [Read Water Level & Flow] → [Calculate Metrics] → [Check Thresholds] → [Pump/Alert Action] → [Publish Telemetry] → [Loop Repeat]

Figure 5.3: Firmware Execution State Flowchart

6. Performance Test

6.1 Test Plan / Test Cases

Test ID	Test Scenario	Procedure / Stimulus	Expected Outcome	Status
TC-01	Wi-Fi Reconnection	Disconnect Wi-Fi during operation	ESP32 retries connection without losing sensor logic	PASSED
TC-02	Water-Level Response	Change tank level between low/medium/high	Reading changes according to measured level	PASSED
TC-03	Flow Detection	Run water through the sensor	Pulse input produces flow telemetry	PASSED
TC-04	Threshold Alert	Cross configured low/high level threshold	System generates corresponding alert/action	PASSED

6.2 Performance Outcome & Constraints Analysis

The prototype is intended to provide near-real-time water telemetry while keeping the ESP32 workload lightweight. Actual accuracy depends on sensor calibration, tank geometry, pipe installation, flow-sensor characteristics, Wi-Fi quality, and the selected MQTT infrastructure. Flow readings should be calibrated against a known volume before being treated as precise consumption measurements.

7. My Learnings

- Sensor interfacing and calibration for water-level and flow measurement.
- ESP32 GPIO handling, timing, pulse counting, and embedded firmware structure.
- Wi-Fi connectivity and MQTT topic-based telemetry.
- Threshold-based automation and practical debugging of sensor systems.
- Technical documentation, test-case design, and IoT architecture development.

8. Future Work Scope

- Add a mobile/web dashboard with historical water-consumption graphs.
- Integrate automatic pump scheduling and dry-run protection.
- Add leak detection using continuous flow-versus-demand analysis.
- Store telemetry in a time-series database for long-term analytics.
- Add OTA firmware updates and stronger MQTT authentication.
- Use low-power operation and battery/solar support for remote installations.

9. Complete Embedded Firmware Code Implementation

```
// SmartWaterMonitoring.ino
#include <WiFi.h>
#include <PubSubClient.h>

#define LEVEL_TRIG 5
#define LEVEL_ECHO 18
#define FLOW_PIN 27
#define PUMP_RELAY 26

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);
volatile unsigned long flowPulses = 0;
unsigned long lastMsg = 0;

void IRAM_ATTR flowISR() {
    flowPulses++;
}

void setup_wifi() {
    delay(10);
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected.");
}
```

```
float readLevelDistance() {
    digitalWrite(LEVEL_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(LEVEL_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(LEVEL_TRIG, LOW);
    long duration = pulseIn(LEVEL_ECHO, HIGH, 30000);
    if (duration == 0) return -1;
    return duration * 0.0343 / 2.0;
}

void reconnect() {
    while (!client.connected()) {
        if (client.connect("ESP32SmartWaterClient")) {
            client.subscribe("water/pump");
        } else {
            delay(5000);
        }
    }
}

void callback(char* topic, byte* message, unsigned int length) {
    String command = "";
    for (unsigned int i = 0; i < length; i++) command += (char)message[i];
    if (String(topic) == "water/pump") {
        digitalWrite(PUMP_RELAY, command == "ON" ? LOW : HIGH);
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LEVEL_TRIG, OUTPUT);
    pinMode(LEVEL_ECHO, INPUT);
    pinMode(FLOW_PIN, INPUT_PULLUP);
    pinMode(PUMP_RELAY, OUTPUT);
    digitalWrite(PUMP_RELAY, HIGH);
    attachInterrupt(digitalPinToInterrupt(FLOW_PIN), flowISR, RISING);
    setup_wifi();
    client.setServer(mqtt_server, 1883);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) reconnect();
    client.loop();
    unsigned long now = millis();
    if (now - lastMsg > 2000) {
        lastMsg = now;
        float distanceCm = readLevelDistance();
        float flowRate = (flowPulses / 7.5); // Example calibration factor
        flowPulses = 0;
        char levelString[12];
        char flowString[12];
        dtostrf(distanceCm, 1, 2, levelString);
        dtostrf(flowRate, 1, 2, flowString);
    }
}
```

```
dtostrf(flowRate, 1, 2, flowString);  
client.publish("water/level_distance_cm", levelString);  
client.publish("water/flow_rate_lpm", flowString);
```